B.Tech. Sem-V

Fundamentals of Operating System Design (CEUC301)

UNIT 7: CONCEPTUAL QUESTION BANK

File-System Design and Storage Allocation

Unit	Unit 7
Question type	Conceptual, analytical and scenario-based

Syllabus coverage: Files, file systems, directories, inodes and inode blocks; boot block, superblock and data blocks; contiguous, linked, indexed and multi-level indexed allocation; free-space management using bitmaps, linked lists, grouping and counting; journaling and log-structured file systems.

A. Files, Directories and File-System Structures

1. Define a file and explain how a file system provides a logical abstraction over persistent storage.
2. Differentiate a file, a directory and a file system. How do they relate to one another?
3. What responsibilities does a directory perform, and why is a hierarchical directory structure useful?
4. Explain the role of an inode in a Unix-like file system. Why is a filename generally stored in a directory entry rather than in the inode itself?
5. What kinds of metadata are typically maintained in an inode, and how does this metadata support file access and management?
6. Explain the conceptual difference between an inode and the data blocks that contain a file's contents.
7. What is meant by inode blocks, and how do file systems organize and locate inode records on storage?
8. Explain the purpose of a boot block and how it differs from ordinary file data blocks.
9. What information is commonly stored in a superblock, and why is it critical to mounting and managing a file system?
10. Differentiate metadata blocks and data blocks. Why can corruption of either category affect file availability or integrity?
11. How can directory organization and inode-based metadata support hard links? How does this differ from a symbolic link?
12. Why must a file system maintain consistency between directory entries, inode metadata, allocation information and data blocks?

B. File Allocation: Contiguous, Linked and Indexed

13. Explain contiguous file allocation and discuss its advantages for sequential and direct access.
14. Why can contiguous allocation suffer from external fragmentation and difficulty growing files?
15. Explain linked allocation and how pointers connect a file's data blocks.
16. What are the performance and reliability limitations of linked allocation, particularly for direct access and pointer corruption?
17. Explain indexed allocation and how an index block enables access to a file's data blocks.
18. Compare contiguous, linked and indexed allocation in terms of sequential access, direct access, metadata overhead and file growth.
19. Why can indexed allocation be more flexible than contiguous allocation while requiring additional storage for index information?
20. How does the choice of allocation strategy affect recovery from a damaged block or corrupted pointer?

21. A workload contains large sequential files and frequent random reads. Compare allocation strategies that might suit this workload and justify your choice.
22. Why is there no single file-allocation method that is optimal for every workload and storage device?

C. Index Allocation and Multi-Level Indexing

23. Why might a single index block be insufficient for a very large file?
24. Explain how multi-level index allocation extends the addressable size of a file.
25. Compare direct pointers, single-indirect pointers, double-indirect pointers and triple-indirect pointers conceptually.
26. What trade-offs does multi-level indexing introduce between maximum file size, metadata overhead and access latency?
27. Why can small files incur disproportionate metadata overhead in an indexed allocation system?
28. How can inode designs combine direct and indirect pointers to balance small-file efficiency and large-file support?
29. Explain why accessing a data block through multiple levels of indirection can require additional storage reads, and how caching can reduce the cost.
30. A file system must support both many small files and a few extremely large files. Explain how a multi-level inode structure can accommodate both patterns.

D. Free-Space Management

31. Why must a file system maintain explicit information about free disk blocks?
32. Explain bitmap-based free-space management and how a bit represents the allocation state of a block.
33. What are the advantages and limitations of bitmaps in terms of search speed, memory overhead and finding contiguous free blocks?
34. Explain linked-list free-space management and discuss its simplicity and limitations when searching for many free blocks.
35. What is grouping in free-space management, and how can it reduce the number of accesses needed to locate free blocks?
36. Explain counting as a free-space management technique and why it works well when free blocks occur in contiguous runs.
37. Compare bitmap, linked-list, grouping and counting methods in terms of metadata overhead, search efficiency and suitability for different allocation patterns.
38. How can a file system keep free-space metadata consistent when blocks are allocated, freed or recovered after a crash?
39. What problems can arise if a block is mistakenly marked free while still referenced by a file, or marked allocated when it is actually unused?
40. A storage volume contains long runs of contiguous free blocks. Which free-space representation could describe them compactly, and why?

E. Journaling and Log-Structured File Systems

41. What problem does a journaling file system attempt to solve, and how does a journal support recovery after a crash?
42. Explain the high-level sequence of recording a transaction in a journal before applying the corresponding file-system updates.
43. Differentiate metadata journaling from data journaling. What consistency and performance trade-offs does each provide?

44. Why does journaling improve crash recovery without necessarily guaranteeing that every recently written application-data byte has been preserved?
45. What is a log-structured file system, and how does it organize updates differently from a conventional in-place-update file system?
46. Why can writing updates sequentially to a log improve performance for some workloads?
47. What challenges does a log-structured file system face when reclaiming space, including segment cleaning and relocating live data?
48. Compare journaling and log-structured file systems in terms of update strategy, recovery, write amplification and space-management overhead.
49. Why are journaling and log-structured designs not synonymous, even though both use log-like structures?
50. A system must recover quickly after unexpected power loss while maintaining good performance for small metadata updates. Explain which file-system properties and journaling choices should be evaluated.

F. Integrative and Higher-Order Conceptual Questions

51. A file can be listed in a directory but cannot be opened. Explain which relationships among directory entries, inode metadata, permissions and data blocks should be investigated.
52. A file system has many small files and a few very large files. Recommend allocation and inode-indexing design choices that balance metadata overhead and scalability.
53. A disk has sufficient total free space, but a large contiguous file allocation fails. Explain how allocation strategy and free-space representation contribute to this situation.
54. After a sudden power loss, directory metadata and allocation information disagree. Explain how journaling can help restore file-system consistency and identify its limits.
55. A workload consists mainly of small random updates. Compare a conventional journaling file system with a log-structured design, considering write patterns and cleaning overhead.
56. Design a conceptual test plan to evaluate a file system for sequential throughput, random access, small-file operations, crash recovery and metadata integrity.
57. Explain how inode design, block allocation, free-space management and crash-recovery mechanisms interact to determine overall file-system reliability.
58. Critically evaluate the statement: 'The fastest file-allocation method is always the best choice.' Consider fragmentation, file growth, metadata cost, reliability and workload characteristics.